Lab 7.1: Asynchronous and Parallel Workflows (F#)

lab7.1.fsx

```
open System.Diagnostics

let stopwatch = Stopwatch.StartNew()

let logMsg (msg: string) =
    printfn "[%4d ms] %s" stopwatch.ElapsedMilliseconds msg

let task (name: string) (delayMs: int) (value: int) : Async<string * int> =
    async {
        logMsg (sprintf "%s started" name)
        do! Async.Sleep delayMs
        logMsg (sprintf "%s finished" name)
        return (name, value)
    }

printfn "=== Async.Parallel ==="
let parallelResults =
    [ task "Download file" 1500 10
      task "Query database" 1000 20
      task "Compute report" 500 30 ]
    |> Async.Parallel
    |> Async.RunSynchronously

printfn "\nResults (in the order tasks were listed):"
parallelResults |> Array.iter (fun (name, value) -> printfn "  %s -> %d" name value)
printfn "Total time: %d ms (sequential would take about 3000 ms)" stopwatch.ElapsedMilliseconds

printfn "\n=== Async.StartChild ==="
stopwatch.Restart()
let childResults =
    async {
        let! a = Async.StartChild(task "Task A" 800 1)
        let! b = Async.StartChild(task "Task B" 400 2)
        let! c = Async.StartChild(task "Task C" 200 3)
        let! ra = a
        let! rb = b
        let! rc = c
        return [ ra; rb; rc ]
    }
    |> Async.RunSynchronously

printfn "\nResults:"
childResults |> List.iter (fun (name, value) -> printfn "  %s -> %d" name value)
printfn "Total time: %d ms (sequential would take about 1400 ms)" stopwatch.ElapsedMilliseconds
```